

$$\text{Location} = \{W_i, G_i, R_i\}$$

Problem 1.1

Alice

$$W_A \xrightarrow{d>0 : \text{take}_A} G_A$$

$$W_A \xrightarrow{d=0 : \text{refund}_A} R_A$$

Bob

$$W_B \xrightarrow{d>0 : \text{take}_B} G_B$$

$$W_B \xrightarrow{d=0 : \text{refund}_B} R_B$$

Reachable states

$$S_0 = \langle \langle W_A, W_B \rangle, d=1 \rangle$$

$$S_1 = \langle \langle G_A, W_B \rangle, d=0 \rangle$$

$$S_2 = \langle \langle W_A, G_B \rangle, d=0 \rangle$$

$$S_3 = \langle \langle G_A, R_B \rangle, d=0 \rangle$$

$$S_4 = \langle \langle R_A, G_B \rangle, d=0 \rangle$$

Unreachable state

$$\langle \langle G_A, G_B \rangle, d=0 \rangle$$

Reachable transitions

$$S_0 \xrightarrow{\text{take}_A} S_1$$

$$S_0 \xrightarrow{\text{take}_B} S_2$$

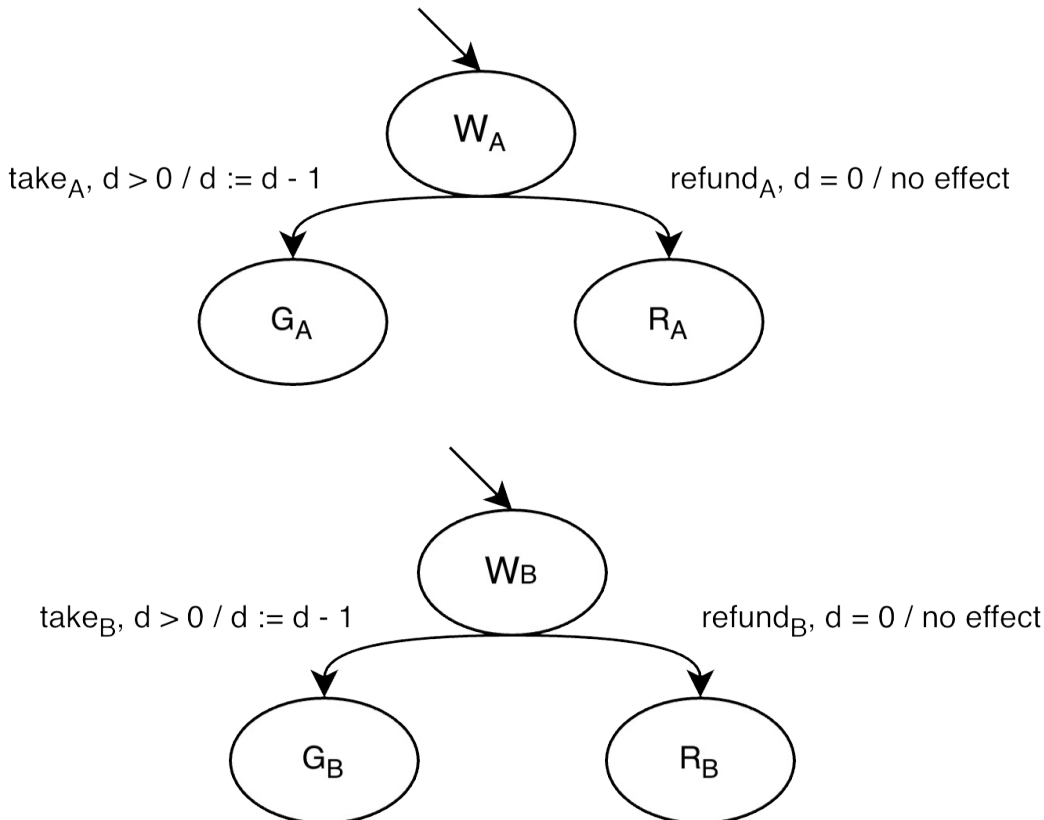
$$S_1 \xrightarrow{\text{refund}_B} S_3$$

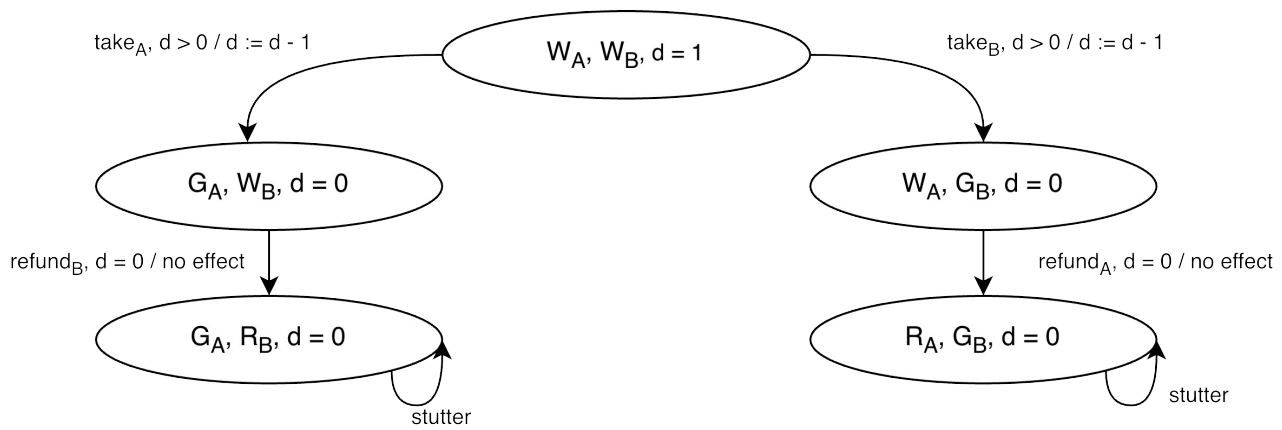
$$S_2 \xrightarrow{\text{refund}_A} S_4$$

Self-loop for terminal states

$$S_3 \xrightarrow{\text{stutter}} S_3$$

$$S_4 \xrightarrow{\text{stutter}} S_4$$





Problem 1.2

#1: Alice gets donut first

$$\begin{aligned}
 \langle \langle W_A, W_B \rangle, d=1 \rangle &\rightarrow \langle \langle G_A, W_B \rangle, d=0 \rangle \rightarrow \langle \langle G_A, R_B \rangle, d=0 \rangle \rightarrow \langle \langle G_A, R_B \rangle, d=0 \rangle \\
 \phi &\quad \{ \text{got } A \} \quad \{ \text{got } A, \text{ref } B \} \quad " \\
 &\phi \{ \text{got } A \} \{ \text{got } A, \text{ref } B \}^\omega
 \end{aligned}$$

#2: Bob gets donut first

$$\begin{aligned}
 \langle \langle W_A, W_B \rangle, d=1 \rangle &\rightarrow \langle \langle W_A, G_B \rangle, d=0 \rangle \rightarrow \langle \langle R_A, G_B \rangle, d=0 \rangle \rightarrow \langle \langle R_A, G_B \rangle, d=0 \rangle \\
 &\phi \{ \text{got } B \} \{ \text{ref } A, \text{got } B \}^\omega
 \end{aligned}$$

$$\text{Trace}(T_{P_A \parallel P_B}) = \{ \phi \{ \text{got } A \} \{ \text{got } A, \text{ref } B \}^\omega, \phi \{ \text{got } B \} \{ \text{ref } A, \text{got } B \}^\omega \}$$

2 infinite traces

Problem 1.3

$$\text{LASTDONUT} = \{ A_1 A_2 A_3 \dots \in (2^{AP})^\omega \mid \forall i \geq 1, \neg (\text{got } A \in A_i \wedge \text{got } B \in A_i) \}$$

$$T_{P_A \parallel P_B} \models \text{LASTDONUT} \text{ because no position contains } \{ \text{got } A, \text{got } B \}$$

Problem 1.4

1. Yes, LASTDONUT is invariant. No reachable state where both Alice and Bob are in their G locations. Invariant holds.

$$\Phi = \neg (\text{got } A \wedge \text{got } B)$$

2. Yes, LASTDONUT is a safety property.

The bad event is $(\text{got } A \wedge \text{got } B)$. If this bad event ever happens at some trace position, then we immediately know the property is violated. Don't have to wait for infinite trace.
 $\therefore$ Every violation has a finite bad prefix.

Problem 1.5

$$\text{NO_REFUND_AFTER_GET} = \{A_1 A_2 A_3 \dots \in (2AP)^{\omega} \mid \forall i > 1, \\ (\text{got } A \in A_i \Rightarrow \forall j \geq i, \text{ref } A \notin A_j) \\ \wedge (\text{got } B \in A_i \Rightarrow \forall j \geq i, \text{ref } B \notin A_j) \}$$

For every position i , if ALICE got A at the position, then from the position i th onwards $\text{ref } A$ should never appear. Same for Bob

$$T_{P_A \parallel P_B} \models \text{NO_REFUND_AFTER_GET}$$

- Possible traces are $\emptyset \{ \text{got } A \} \{ \text{got } A, \text{ref } B \}^{\omega}$ and $\emptyset \{ \text{got } B \} \{ \text{ref } A, \text{got } B \}^{\omega}$
 - In the first trace, Alice gets the donut and $\text{ref } A$ never appears
 - In the second trace, Bob gets the donut and $\text{ref } B$ never appears too.
- $\therefore$ The property holds

1) - No, this not an invariant over the current AP.

- It depends on history.
- Current $\text{ref } A$ label is only bad if $\text{got } A$ happened earlier.
- So, it cannot be checked only from the current label.

2) - Yes, it is a safety property.

- Any violation has a finite bad prefix.
- Once $\text{got } A$ occurs and $\text{ref } A$ later occurs, (or same with B), violation is already visible in a finite prefix.

Problem 1.6

- Using pure-interleaving for two transition systems that use a shared variable, d will lead to incorrect TS-level composition of $T_{PA} \parallel T_{PB}$
- If they are unfolded separately, Alice's local transition system has its own valuation of d as well as Bob too.
- Therefore, both local systems may start with $d=1$.

In the incorrect model, these paths become reachable:

$$\langle \langle W_A, d_A=1 \rangle, \langle W_B, d_B=1 \rangle \rangle \xrightarrow{\text{take}_A} \langle \langle G_A, d_A=0 \rangle, \langle W_B, d_B=1 \rangle \rangle \xrightarrow{\text{take}_B} \langle \langle G_A, d_A=0 \rangle, \langle G_B, d_B=0 \rangle \rangle$$

- This reaches a state where Alice is in G_A and Bob is in G_B . The label becomes $\{\text{got}_A, \text{got}_B\}$
- This state should not be possible in the real shared-variable model
- Only one shared d is allowed in the correct model. When Alice performs take_A , shared value becomes $d=0$ and Bob performs ref_B and vice versa.

Problem 2.1

Pure interleaving $T_A \parallel T_B$ = Alice and Bob move independently.

9 reachable states

$$\{A_0, A_1, A_2\} \times \{B_0, B_1, B_2\} = 9$$

$$\begin{array}{ccc} (A_0, B_0) & (A_0, B_1) & (A_0, B_2) \\ (A_1, B_0) & (A_1, B_1) & (A_1, B_2) \\ (A_2, B_0) & (A_2, B_1) & (A_2, B_2) \end{array}$$

Path where Alice becomes done before Bob is.

$$(A_0, B_0) \xrightarrow{\text{find}_A} (A_1, B_0) \xrightarrow{\text{unlock}} (A_2, B_0) \xrightarrow{\text{find}_B} (A_2, B_1) \xrightarrow{\text{unlock}} (A_2, B_2)$$

Path trace:

$$\emptyset \{ \text{ready}_A \} \{ \text{done}_A \} \{ \text{done}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^w$$

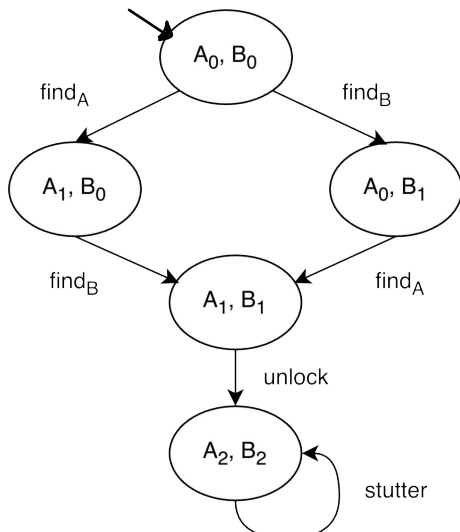
Problem 2.2

$T_A \parallel_{\text{sync}} T_B$ where unlock must be performed by Alice and Bob simultaneously

5 reachable states

Reachable transitions

$$\begin{array}{l} (A_0, B_0) \\ (A_1, B_0) \\ (A_0, B_1) \\ (A_1, B_1) \\ (A_2, B_2) \end{array} \quad \begin{array}{l} (A_0, B_0) \xrightarrow{\text{find}_A} (A_1, B_0) \\ (A_0, B_0) \xrightarrow{\text{find}_B} (A_0, B_1) \\ (A_1, B_0) \xrightarrow{\text{find}_B} (A_1, B_1) \\ (A_0, B_1) \xrightarrow{\text{find}_A} (A_1, B_1) \\ (A_1, B_1) \xrightarrow{\text{unlock}} (A_2, B_2) \end{array} \quad \text{Stutter self-loop: } (A_2, B_2)$$



States from pure interleaving that become unreachable:

$$\begin{array}{l} (A_2, B_0) \\ (A_2, B_1) \\ (A_0, B_2) \\ (A_1, B_2) \end{array}$$

$\therefore$ Alice and Bob need to perform unlock together.

Problem 2.3

$$\emptyset \{ \text{ready}_A \} \{ \text{ready}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^\omega$$

$$\emptyset \{ \text{ready}_B \} \{ \text{ready}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^\omega$$

2 possible infinite traces where first Alice finds the key first and the second one is Bob finds the key first.

Problem 2.4

$$\text{SIMUL_UNLOCK} = \{ A_1 A_2 A_3 \dots \in (2^{AP})^\omega \mid \forall i \geq 1, (\text{done}_A \in A_i \Leftrightarrow \text{done}_B \in A_i) \}$$

At every position i , done_A is true if and only if done_B is also true.

For pure interleaving model,

$$T_{\text{pure}} \not\models \text{SIMUL_UNLOCK}$$

Counterexample trace: $\emptyset \{ \text{ready}_A \} \{ \text{done}_A \} \{ \text{done}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^\omega$

- done_A is true but done_B is false in the third sequence.
- $\therefore$ violates SIMUL_UNLOCK .

For sync model:

$$T_{\text{handshaking}} \models \text{SIMUL_UNLOCK}$$

- the only possible traces are $\emptyset \{ \text{ready}_A \} \{ \text{ready}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^\omega$
and $\emptyset \{ \text{ready}_B \} \{ \text{ready}_A, \text{ready}_B \} \{ \text{done}_A, \text{done}_B \}^\omega$
- In both traces, done_A and done_B always appear together.
- $\therefore$ Sync satisfies the SIMUL_UNLOCK .

Problem 2-5

① - Yes `SIMUL_UNLOCK` is an invariant.

- It can be checked at each individual state using the state formula :

$$(doneA \wedge doneB) \vee (\neg doneA \wedge \neg doneB)$$

- This requires Alice and Bob to be either both done or both not done at every state

② - Yes `SIMUL_UNLOCK` is a safety property.

- A violation occurs as soon as some finite trace prefix reaches a position where `doneA` is true but `doneB` is false or `doneB` is true but `doneA` is false.
- Such violation can be detected from a finite prefix so `SIMUL_UNLOCK` is a safety property.